LAB ASSIGNMENT 4–5

Advanced MLflow and Data Versioning with DVC

1. Objective

The objective of this assignment is to understand machine learning experiment tracking, model comparison, model registration, dataset versioning, and reproducibility. MLflow is used to track experiments and manage model versions. Git tracks source code and DVC metadata, while DVC stores and versions dataset contents.

2. Tools and Technologies

- Python 3.11
- MLflow 3.15.1
- DVC 3.67.1
- Scikit-learn 1.9.0
- Pandas
- Git

3. Dataset and Methodology

A small synthetic house-price dataset is used. The input features are area in square feet, number of bedrooms, age in years, and location score. The target variable is house price.

The dataset initially contained 30 records. Three additional records were added to create an updated version containing 33 records.

The data was split into training and testing sets using an 80:20 split and `random_state=42`. Three Random Forest Regressor configurations were trained and evaluated.

4. MLflow Experiment Tracking

MLflow was used to record model parameters, evaluation metrics, dataset SHA-256, and Scikit-learn version. Model artifacts and model signatures with input examples were logged.

The three model configurations and their results are shown below.

Model	MAE	RMSE	R^2

RF_50_Trees	14193.33	16215.89	0.9567
RF_100_Trees	13050.83	14615.08	0.9648
RF_100_Trees_Depth10	12991.67	14668.72	0.9646

MAE and RMSE measure prediction error; lower values are generally better. R^2 measures how well the model explains variation in the target; a value closer to 1 is generally better.

The 100-tree model with maximum depth 5 achieved the highest R^2 score among these three experiments. The depth-10 model had a slightly lower MAE, illustrating why multiple metrics should be compared.

5. Model Registration and Lifecycle

The trained models were registered under the MLflow model name `HousePriceModel`. Three model versions were created.

Lifecycle tags were used to demonstrate model management: Version 1 was tagged `archived`, Version 2 `staging`, and Version 3 `production`. The `champion` alias was assigned to Version 3.

These lifecycle labels demonstrate registry operations and should not be interpreted as proof that Version 3 has the best evaluation performance.

6. Dataset Versioning with DVC

DVC was initialized to manage the dataset. Git tracks the `.dvc` metadata file, while DVC stores the dataset contents in its cache and configured remote.

Two versions were verified:

- Version 1: 30 records, 631 bytes.
- Version 2: 33 records, 693 bytes.

The DVC remote was configured as a local folder on the development machine. The `dvc push` command completed successfully, and `dvc status` reported that data and pipelines were up to date.

A recovery demonstration restored Version 1, and the dataset was verified to contain 30 rows. Version 2 was then restored and verified to contain 33 rows.

7. Git and MLflow Integration

Git was used to track project files and DVC metadata. Git history contains commits for project setup, DVC initialization, dataset tracking, and the updated dataset and model files.

The three existing MLflow runs were tagged with the Git commit hash and the SHA-256 hash of the original 30-row dataset used for those training runs. This connects the recorded experiments to their dataset identity and code revision.

8. Reproducibility

The `README.md` describes project setup, training, MLflow, and DVC commands. The `requirements.txt` lists the principal dependencies.

The main workflow is:

1. Install the listed dependencies.
2. Restore the dataset version associated with the chosen Git revision using DVC.
3. Run `python src/train.py`.
4. Start the MLflow UI and compare the experiment results.
5. Inspect model versions and recorded artifacts.

The configured DVC remote is local to the original development machine. Reproduction on another machine requires access to a suitable remote containing the DVC data.

9. Challenges Faced

During the assignment, dataset versions had to be restored carefully to avoid losing the updated data. Git and DVC were used together to recover the original dataset and then return to the updated version.

The assignment also demonstrated the importance of checking dataset hashes, verifying DVC status, and recording experiment metadata.

10. Conclusion

This assignment demonstrated experiment tracking and model comparison with MLflow, model registration and lifecycle tagging, and dataset versioning with Git and DVC. The recovery test verified that the original and updated datasets could be distinguished and restored.

The dataset is small and synthetic, so the reported metrics demonstrate the workflow rather than real-world house-price prediction performance.

11. Screenshot Evidence

The `reports/figures` folder contains screenshots of dataset verification, DVC metadata and push, DVC status and recovery, MLflow training results, the experiment interface, run

details, and model versions. These screenshots provide evidence of the implemented workflow.

12. References

- MLflow documentation: https://mlflow.org/docs/latest/
- DVC documentation: https://dvc.org/doc
- Scikit-learn documentation: https://scikit-learn.org/stable/
- Git documentation: https://git-scm.com/doc